Problem Statement

Objective

In this case study, you will explore the fundamental components of a Retrieval-Augmented Generation (RAG) system using legal text data. The focus will be on applying semantic search to retrieve and synthesise information from a corpus of legal documents.

You will learn to implement key stages of a basic RAG pipeline, including document loading, creating an embedding layer, using a vector database for storing embeddings, and the integration of retrieval with language generation models. Along with this, you will utilise GenAI Frameworks like LangChain or LlamaIndex. These frameworks provide developers with the necessary tools and abstractions that reduce the development time for building AI applications as well as developing LLM-agnostic applications.

The aim is to develop practical skills in combining structured retrieval techniques with unstructured language generation to support intelligent information access in a legal context.

Business Value

Legal firms and departments often handle large volumes of complex documents, making it difficult to efficiently find specific information. A streamlined RAG-based solution provides quicker and more precise responses to legal queries by dynamically retrieving and contextualising relevant excerpts from a legal corpus.

By adopting this approach, legal professionals can save time on document review, enhance research accuracy, and make more informed decisions. By adopting this approach, legal professionals can save time on document review, enhance research accuracy, and make more informed decisions.

Dataset Overview

Context

The dataset contains legal documents and contracts collected from various sources. The data includes a variety of legal documents such as privacy policies, merger agreements, and non-disclosure agreements. Using these, we can build upon a base of precedents and domain knowledge.

Content

The documents are present as text files (.txt) in the corpus folder. There are four types of documents in the corpus folder, divided into four subfolders.

- `contractnli`: contains various non-disclosure and confidentiality agreements
- `cuad`: contains contracts with annotated legal clauses
- `maud`: contains various merger/acquisition contracts and agreements
- `privacy_qa`: a question-answering dataset containing privacy policies

The dataset also contains evaluation files in JSON format in the benchmark folder. The files contain the questions and their correct answers, along with sources. For each of the above four folders, there is a json file: `contractnli.json, cuad.json, maud.json, privacy_qa.json`.
The file structure is as follows:

```
{
    "tests": [
        {
            "query": <question1>,
            "snippets": [{
                    "file_path": <source_file1>,
                    "span": [ begin_position, end_position ],
                    "answer": <relevant answer to the question 1>
                },
                {
                    "file_path": <source_file2>,
                    "span": [ begin_position, end_position ],
                    "answer": <relevant answer to the question 2>
                }, ....
            ]
        },
        {
            "query": <question2>,
            "snippets": [{<answer context for que 2>}]
        },
        ... <more queries>
    ]
}
```

Scoring and Penalty

- **Total Marks**: **50** (for the code notebook)
- **Extension and Penalty**: As given in your learner handbooks

Instructions

1. Each learner should attempt this assignment individually.
2. Programming Language: Python
3. You will be provided with the dataset and a starter notebook. You have to work in the starter notebook only.
4. It is very important that you do not change any headings, subheadings, questions or tasks in your notebook as it can cause problems with grading.
5. If you find any inconsistencies and outliers, please handle them as per your understanding and mention them in your solution.
6. You are encouraged to search the web and consult AI tools for conceptual understanding. However, using plagiarized or AI-generated code is strictly prohibited and strongly discouraged.
7. Submitting plagiarized and AI-generated code or reports will result in significant penalties to your scores.

Submission Guidelines

1. To submit your solution, upload your solution file in the submission field.
2. You are required to upload your solution as an IPYNB file titled `"RAG_Legal_Docs_<your_name>.ipynb"`.
3. The Interactive Python Notebook (.ipynb) should contain your code and your visualisations, analysis, results, insights, and outcomes.

4. Note that this file should only be generated from the starter files provided to you.
5. The Jupyter notebook should contain your name and the assignment title.
6. Mention all assumptions made.

Results Expected from Learners

In the starter notebook, you will find headings, subheadings, and checkpoints stating the tasks you need to perform. The marks associated with each checkpoint will also be mentioned in the notebook. Keep in mind not to edit the cells with marking schemes and questions. You can find a brief description of the tasks below.

1. Data Loading, Preparation and Analysis [20 marks]

1.1 **Data Understanding**
- Understand the data for documents and queries and associated answers.

1.2 **Load and Preprocess the data** [5 Marks]
i. Load all '.txt' files from the folders.
ii. Preprocess the text data to remove noise and prepare it for analysis.

1.3 **Exploratory Data Analysis** [10 Marks]
i. Calculate the average, maximum and minimum document length
ii. Analyse the frequency of occurrence of words and find the most and least occurring words.
iii. Analyse the similarity of different documents to each other based on TF-IDF vectors.

1.4 **Document Creation and Chunking** [5 Marks]
i. Perform appropriate steps to split the text into chunks.

2. Vector Database and RAG Chain Creation [15 marks]

2.1 **Vector Embedding and Vector Database Creation** [7 marks]
i. Initialise an embedding function for loading the embeddings into the vector database.
ii. Load the embeddings to a vector database.

2.2 **Create a RAG chain** [8 marks]
i. Create a RAG chain.
ii. Create a function to generate an answer for the asked questions.

3. RAG Evaluation [10 marks]

3.1 **Evaluation and Inference** [10 marks]
i. Extract all the questions and all the answers/ground truths from the benchmark files.
ii. Evaluate retrieval and generation.
iii. Draw inferences by evaluating answers to a sample of the questions.

4. Conclusion [5 marks]

4.1 **Conclusions and insights** [5 marks]

Evaluation Rubrics

The following rubrics will be used while judging your solutions to the above tasks.

Table 1: Rubrics

Criteria	Evaluation Parameters
Overall System Design	1. Optimum system architecture, workflow and implementation 2. Appropriate use of the components of LangChain/LlamaIndex in the system design 3. Logical integration of different components (data processing, vector DB, RAG chain).
Data Preparation	1. Completeness and correctness in loading, cleaning, and preprocessing data from text files. 2. Demonstrated understanding of the nature and structure of documents and queries. 3. Insightful and technically sound exploratory data analysis, including document statistics and similarity analysis. 4. Appropriate and justified document chunking strategy to prepare data for embedding.
Vector DB and RAG Chain Setup	1. Correct and justified use of embedding models for generating document vectors. 2. Correct implementation and setup of a vector database with a proper and accessible persist directory/storage. 3. Functional and logically constructed RAG pipeline, with proper retrieval and generation steps. 4. Robust implementation of a query-answering function integrating a retriever and a generator.
Evaluation	1. Correct and justified use of embedding models for generating document vectors. 2. Efficient setup of a vector database with a clear rationale for the chosen technology. 3. Functional and logically constructed RAG pipeline, with proper retrieval and generation steps. 4. Robust implementation of a query-answering function integrating a retriever and a generator.